

ly used under the following circumstances, when the merge request is especially urgent. For example:

- Resolves or mitigates a severity::1/severity::2 incident
- Resolves a regression that can lead to a severity::1/severity::2 problem
- Urgent performance or availability fix that can improve the stability of GitLab.com

In case of high-severity bugs, please proceed to report an incident to track events and response.

If this label is added because a merge request is blocking further deploys, consider leaving a note in releases Slack channel to raise awareness of the status.

For new features or non-urgent fixes, the label should not be used because the new release is only days or hours away.

Directions on how to know whether a MR is deployed to GitLab.com are in the release/docs.

Frequently Asked Questions

When will my merge request be deployed?

We currently create an auto-deployment branch at a specific timeline. The current Mean Time To Production time and target can be seen on the Infrastructure performance indicators page.

For a merge request with the pick label, the process is different.

How can I determine what environment my merge request is currently in?

The release tools makes use of workflow:: labels in MRs to indicate which stage of the deployments process a merge request is in. For example, the presence of the workflow::production label in a merge request indicates it has been deployed to production.

See this guide for more information.

I found a regression, what do I do next?

If you've found a regression with a potentially high severity, immediately follow the steps in Deployment blockers to halt the deployment.

For high severity bugs found in the lead up to the monthly release please also alert the Release Managers in releases.

Resources

Description	Location	
-------------	----------	--

|-----|-----|
| Deployment orchestration | Link |
| Deployment documentation | Link |
| Release related tasks issue tracker | Link |
| Delivery group issue tracker | Link |
| Release manager schedule | Link |
| Maintenance Policy | Link |

SECTION: engineering/development/_index.md

SECTION: engineering/development/database.md

This page is dedicated to database application engineering and provides an entry-point for resources on this topic.

Also see Database Team in Enablement.

GitLab development

Please refer to the development documentation for database guidelines.

Database Roles at GitLab

We have two primary job roles that are focused on the database aspect:

1. Backend Engineer, Database - in Development
2. Database Reliability Engineer - in Infrastructure

The Backend Engineer, Database role is a software engineering role concentrated on application-side improvements and foundational database work in the GitLab codebase.

The Database Reliability Engineer is an operational role targeting and running the database infrastructure for GitLab.com from within the Reliability Engineering teams.

Database Maintainer

For the GitLab codebase, Database Maintainers contribute to the code review process by reviewing database-related changes and applying database review guidelines. They typically engage into conversations about database queries and their performance, database schema design and database migrations.

The Database Maintainer role:

Is an additional role, typically for Backend Engineers.

Follows the same definition as any other GitLab maintainer.
It does not come with operational responsibility for GitLab.com and its database infrastructure. This responsibility belongs to the Database Reliability Engineer.
It is not expected to participate in incident management issues.
Adheres to the review turnaround time of 2 working days.

If you're interested in participating in database reviews, please start by reviewing the Database maintainer process which contains all the resources for a reviewer.

Recommended links and reference materials

GitLab resources

Database Office Hours are bi-weekly and can be found on the GitLab Team Meetings Calendar
Database Office Hours playlist on GitLab Unfiltered under the Database Office Hours Playlist

Books

(In no particular order.)

I. Ahmed, G. Smith et al: "PostgreSQL 10 High Performance: Expert techniques for query optimization, high availability, and efficient database maintenance." (2018)
Hans-Jürgen Schönig: "Mastering PostgreSQL 11" (2018)
Markus Winand: "SQL Performance Explained"
Dimitri Fontaine: "The Art of PostgreSQL" - Virtual training is available for GitLab team members.
Alex Petrov: "Database Internals"

Other resources

Official PostgreSQL mailinglists, particularly pgsql-general
Postgres Weekly
Planet PostgreSQL
SQL Indexing and Tuning e-Book (also known as "Use The Index Luke")
Weekly show on Scaling PostgreSQL
Detailed documentation of the operations and fields in PostgreSQL EXPLAIN query plans
Weekly podcast about all things PostgreSQL postgres.fm

SECTION: engineering/development/managers.md

This page is for any development manager notes we want to share more broadly. A directory was created in case we need to archive or break out content.

FY21 Compensation Review

The following is a brief for engineering managers to help in their conversations with team members about compensation. This document is to give additional context on the FY'21 compensation plan and how GitLab views compensation. It should be viewed as complementary to

the handbook information.

Background

As of FY'21 GitLab evaluates total compensation when determining team members's compensation package. Total compensation is defined as Salary pay + Stock Equity + Benefits and is guided by the principles defined here. Salary pay is determined by local markets and cost of market. Stock Equity (if available for a team member's position) is determined by role and seniority. Equity is uniform across the world. Increases to budget for FY'21 are determined across two factors. A 6% increase in Salary review and a 1.2%(12% headcount 10% salary bump) increase for promotions. Please note the budget for promotions is a budgetary consideration, promotion % increases may vary. The Salary review is done twice a year with the budget being increased 5.28% effective February 1st and 0.72% effective August 1st. The promotion review is currently continuous. The approximate amount spent on location factor to bring people in band this year for development was 0.89%.

For SSOT on these calculations and full detail, please refer to this page.

Goals of FY'21 Compensation Review

The goals for FY'21 compensation for the development department in priority order are as follows:

1. NOT reduce anyone's salary
1. Stay within the prescribed budgetary constraints listed above, or ask for additional funding if our goals are not achievable.
1. Ensure everyone is within their compensation range (as defined in the compensation calculator available to all GitLab team members).
1. Give increases based on current performance factor as evaluated during the FY21 end of year review.
1. Give managers flexibility to consider location factor, performance, recent promotions and recent compensation adjustments in rewarding employees.
1. Conduct a comprehensive at the VP/PBP/Total Rewards level so that pay increases eliminate unfairness where possible across the organization

FAQ

1. Question: Due to the removal of contractor factor some exceeding team members are above the top of their range. Did we limit performance-based increases for these team members? Will it be more difficult for these individuals to get performance-based increases in future cycles?

1. Answer: While being at the top of the band does indicate that these team members are paid fairly against the market, we are open to reviewing business justification cases for these team members as applicable. There are a variety of factors that contribute to increases (benchmark, location factor, performance, last promotion/raise, etc.) so these situations were managed on a case-by-case basis. In terms of future increases, it would be hard to predict the future here as we iterate on the inputs at least once a year meaning if an over range team member doesn't receive a promotion, depending on future inputs they may move back into the range (no guarantees of course).

1. Question: Why is the increased budget lower than before?

1. Answer: The budget for the annual compensation review was the same as it has been in past

years. This handbook page helps explain the overall annual review budgetary calculation. Total Budget = 6% x 88% x [October OTE USD]. Companies typically allocate 3% to annual comp review, while GitLab allocates 6%.

1. Budgetary discretion this year should have felt better than last year due to the depreciation of compa groups. Last year, Engineering Managers did not get the opportunity to go into Compaas because the budget was more or less depleted getting everyone into their compa group with benchmark and location factor increases. This year is different because there was a skew to higher location factor increases in lower location factor areas, where last year it was more across the board as we added in a new data source.

1. Question: Location factor saw a huge increase in my area but I only see 4% more, why?

1. Answer: Not all location factors increase at the same rate. Total Rewards performs a thorough analysis of all location factors in October/November just ahead of annual comp. If you have questions or feedback about your location factor, please reach out to total-rewards@gitlab.com.

1. Note that in general, "Everywhere else" saw larger increases. For example, if a metro was (purely hypothetical numbers) 0.65 and the "everywhere else" was 0.5, but the geo zone was made 0.6 based on the entire geographical zone, the "everywhere else" may have seen a huge increase in the implementation of geo zones while the metro may have only seen a more 'typical' location factor increase.

1. Question: Is the fact that I'm lower in the band reflective of my performance?

1. Answer: It is important to note that where someone falls within the compensation range is not an indicator of performance, nor is it a sign of promotional readiness. For more information on how performance is linked to compensation please review the following handbook section.

SECTION: engineering/development/okrs.md

The source of truth for GitLab OKRs and KRs for Development is the GitLab OKRs tool. Development objectives and KRs are captured on this page.

FY24Q3

1. OKR Continue to win against GitHub with AI => 66%

1. KR: GA Code Suggestions => 50% - Needs Attention

1. KR: Beta GitLab Duo => 40% - Needs Attention

1. KR: Increase accepted code suggestions rate to 20% => 50% - At Risk

1. KR: Ship Iteration 2 of Self-Managed AI (GitLab Plus) => 100% - On Track

1. KR: Create: Continue to win against GitHub with AI => 64% - On Track

1. KR: Ops: AI Goals =>100% - Needs Attention

1. OKR Reducing Churn and Contraction => 61%

1. KR: Ship temporary licenses capability for self-managed customers => 80% - On Track

1. KR: Reach a 50% minimum per group of pajamas compliance for drop downs => 63% - On Track

1. KR: Verify CI Catalog goal - Beta or GA release => 40% - At Risk

1. OKR Make GitLab easier to do business with => 60%

1. KR: In collaboration with PS migrate 3 new large customers to Dedicated => 60% - On Track

1. KR: Deliver a consolidated SSOT Monorepo guidance => 80% - On Track

1. KR: Project Hamilton Delivery => 70% - On Track

1. OKR Continue to build a diverse team of top talent that we retain and grow => 52%

- 1. KR: Meet new hiring goals => 80% - Needs Attention
- 1. KR: TBD placeholder for actions to employee engagement survey feedback => 35% - On Track
- 1. KR: Create Stage OKRs => 40% - On Track
- 1. OKR Engineering efficiency and foundations => 59%
 - 1. KR: Ship Gitaly enhancements for scalability and disaster recovery => 35% - At Risk
 - 1. KR: PoC on Auth/Auth service => 100% - On Track
 - 1. KR: Ship Cells planned work => 50% - At Risk
 - 1. KR: Ops Engineering Efficiency => 51% - On Track
- 1. OKR Joint Product Goals each groups major deliverables for Q3 => 45%
 - 1. KR: FY24Q3 Fulfillment Product Deliverables => 46% - On Track
 - 1. KR: FY24Q3 - Enablement O6: Joint Product Goals => 62% - Needs Attention
 - 1. KR: (3.0) Create: Joint Product Goals each groups major deliverables => 47% - On Track
 - 1. KR: Sec/Growth/Data Science: Major roadmap deliverables => 37% - On Track
 - 1. KR: Ops: Product Aligned Goals => 47% - On Track
 - 1. KR: Dev + Analytics - Deliver on high priority product roadmap items => 31% - Needs Attention

FY24Q2

- 1. OKR Continue to win against GitHub with AI in all we do => 66%
 - 1. KR: Create: Compete with GitHub => 65%
 - 1. KR: Experimental launch of Workspaces feature used by 10 team members to develop GitLab features. => 70%
 - 1. KR: Ops: Compete with GitHub => 70%
 - 1. KR: Assess and propose organizational structure for AI moving forward (end of quarter) => 40%
 - 1. KR: Development: AI in everything we do => 87%
 - 1. KR: Development: 32 Experimental, 12 Beta and 8 GA AI assisted features => 66%
- 1. OKR Reducing Churn and Contraction => 74%
 - 1. KR: Start adding request to prioritize small fixes to our current milestones => 81%
 - 1. KR: Manage:Foundations - Execute Pajamas Button Migration for 560 Buttons => 68%
- 1. OKR Make GitLab easier to do business with => 73%
 - 1. KR: FY24Q2 - Database: Improve migration reliability => 30%
 - 1. KR: Development: Refresher training on self-managed required stops => 90%
 - 1. KR: FY24Q2 - Enablement O2.4.1: Add examples of breaking changes to the product documentation => 100%
- 1. OKR Continue to build a diverse team of top talent that we retain and grow => 73%
 - 1. KR: Refine the Engineering promotion process => 80%
 - 1. KR: KR: Team member check-ins completed with a growth plan in place with a focus on ensuring business continuity throughout FY24 => 90%
 - 1. KR: KR: 100% of Managers in Engineering complete the Neurodiversity short course in LevelUp => 50%
- 1. OKR Engineering efficiency and foundations => 64%
 - 1. KR: FY24Q2 - Enablement O1.1: Complete Plumbing Work For Disaster Recovery Mid-Long Term Solutions - WAL, Git Bundle, Raft => 85%
 - 1. KR: Create: Engineering Efficiency => 53%
 - 1. KR: Dev: Improve Frontend Engineering Efficiency => 52%
 - 1. KR: Ops Section: Engineering Efficiency => 67%
- 1. OKR Joint Product Goals each groups major deliverables for Q2 => 77%
 - 1. KR: FY24Q2 Fulfillment Product Deliverables => 89%

- 1. KR: FY24Q2 - Enablement O4: Joint Product Goals each groups major deliverables => 71%
- 1. KR: (4.0) Create: Joint Product Goals each groups major deliverables for Q2 => 100%
- 1. KR: Sec/Growth/Data Science: Deliver on high priority product roadmap items =>54%
- 1. KR: Ops - See Compete with GitHub Objective => 69%
- 1. KR: Dev + Analytics - Deliver on high priority product roadmap items => 82%

SECTION: engineering/development/performance-indicators.md

{{% performance-indicators "developmentdepartment" %}}

SECTION: engineering/development/principles.md

Engineering Principles

At GitLab, Company Culture is very important to us. The main ingredient of the company culture are GitLab Values.

GitLab Values have guided us throughout the evolution of the company. Those values have been crucial in maintaining a positive and productive culture, helping us make decisions to make the company and the product better.

Our Engineering Principles are built on top of GitLab Values, and provide additional explanation of these in the context of the software engineering practice.

- 1. Iteration
- 1. Efficiency
- 1. Simplicity
- 1. Reliability
- 1. Quality
- 1. Velocity
- 1. Predictability

Iteration

We always push ourselves to be iterative and make the minimal valuable change that is on the direct path to achieving our goals.

For complex initiatives we are using the Architecture Design Workflow to define our goals and to describe our iteration plan for a given effort. How we are going to iterate on something may depend on our mid-to-long-term vision. Our goal is to make the iteration as efficient and as pragmatic as possible.

We also sometimes use an established "scooter heuristic": release a smallest product that our customers can use, and then iterate on it.

Image Credit: Henrik Kniberg from Crisp

Releasing minimal changes, as quickly as possible, is sometimes a good way to think about iteration, especially when intermediate steps provide value to our users and allow us to collect meaningful feedback. This has often been our default iteration approach.

This "scooter" iteration pattern, however, is not always the most efficient way to make progress on something. Sometimes we do need to have a tailored plan for efficient iteration, because if we want to build something more complex, our users may not be happy with a scooter or a paper airplane, and we will either not get any meaningful feedback or it will not be significant enough to be useful. In such cases, how we iterate should be a diligently planned strategy, designed to improve efficiency and to allow us to achieve our goals within a reasonable amount of time.

At GitLab we can use our Architecture Design Workflow to build a more pragmatic iteration plan aligned with mid-to-long term vision. This usually covers building a Minimal Viable Product too. In this case, however, how the MVP is going to look like depends on a thoughtful plan, described in writing after collecting feedback throughout the organization.

Below you can find a few patterns that can precede "building the smallest thing in the next milestone" pattern, aimed to increase iteration efficiency:

1. Identify fundamental design problems, describe these in a design doc.
1. Identify most significant risks associated with those, find ways to de-risk.
1. Develop a plan to iterate on the fundamental design aspects individually.
1. Deliver a first end-to-end iteration as a prototype or a Proof-of-Concept.

All these things attempt to improve our level of knowledge and reduce implementation risk. All of these steps are also iterations by themselves that can help us find hidden efficiencies to build better MVPs.

To read more about iteration, see our values page and engineering workflow.

Efficiency

Efficiency is at the core our engineering discipline at GitLab. This is also one of the most important company values.

How we work in Engineering can have a significant compounding effect over the mid-to-long term. If we constantly ask ourselves, "How can we implement this thing to become more efficient in mid-to-long term?" while still thinking about pragmatic short-term goals, we can hope for seeing a lot value being built over time, similarly to how compound interest builds wealth. Overfocusing on short-term goals can lead to us missing out on this opportunity.

A short, and incomplete list of things that can help us become more efficient:

1. Build useful and well designed abstractions.
1. Choose simple solution over easy solutions.
1. Refactor code for simplicity and ease of understanding.
1. Contribute to an open source project instead of patching locally.
1. Build automation to reduce repetitive work or simplify a process.
1. Build new communication channels that enhance knowledge sharing.
1. Strive for achieving engineering excellence over engineering perfection.

Simplicity

Simplicity is a prerequisite for reliability. It is also at the core our boring solutions value. We strive for choosing simple solution over easy solution, even when it sometimes means that we will need to put more work into building a simple solution.

Reliability

Availability/Reliability, Quality, Security, and Performance are the pillars for building reliable software. Reliability is our contract with our customers that say you can count on us to deliver an available and dependable product. Everyone in the organization has a role to play.

Engineers, Product Managers, and Designers have the most direct influence over the reliability of the code through either planning, implementation, monitoring (e.g. Kibana, Sentry, Grafana and other GitLab.com monitoring tools), or prioritization of the work. Product and Engineering management monitors (e.g. Error Budgets) and measures the reliability of features and makes recommendations if necessary. Our focus on learning and development will also ensure that teams have the tools and training required to build reliable software. The Infrastructure, Application Security, Database and Developer Experience teams are the Subject Matter Experts supporting product development teams.

Velocity

Our velocity should be incremental in nature. It's derived from our Iteration and Efficiency principles.

We recognize the importance of keeping our velocity high:

- The rate at which GitLab delivers new value to users is a competitive advantage for the company.
- As an open source project, people are welcome to fork us. However, in order to ensure that the community contributing to GitLab remains intact it is important to move fast so that any fork is quickly out-of-date.
- Once you slow down, it is incredibly painful to speed back up again.
- Companies tend to slow down as they grow. It takes deliberate effort to prevent this, so it must always be top of mind. Highlighting the need of retaining velocity can help us with adapting workflows and processes to keep velocity high.

There are a few things that we can do to keep our high velocity. See those described below.

Iterate well

Iterate well on what you are working on. Strive for delivering small merge requests. Write a design docs when you plan to ship multiple iterations that would benefit from additional alignment.

Velocity comes from going fast into the right direction. That is the difference between Speed and Velocity. Direction matters for Velocity, therefore we need to iterate efficiently on the right things at the right time.

For some tips on how to iterate you can check the blog post about 12 shortcuts that are not iteration.

Reduce review time

Be mindful about a code review latency. Do not make authors wait too much before you review their code.

For many people context switching is a very expensive thing to do, which can negatively impact their productivity. Be mindful about that when you are reviewing code or collaborating with team members asynchronously. If a question doesn't need to be answered, or a code suggestion is a minor nitpick which doesn't need to be addressed, then perhaps it is not worth context switching required to do these things.

Reduce process

Reduce the amount of attention to various processes when working on Experimental and Beta features. It is fine to skip certain processes when you are aiming for delivering an Experimental or even Beta feature. Processes can also be automated to reduce their impact on velocity.

For example, if you are working on an Experimental feature, you should aim for reducing the amount of time-consuming processes to the bare minimum.

At GitLab we do not ship the process.

This might require a pragmatic approach to find the right balance between working on code and doing UX research, customer validation or spending time in long brainstorming sessions. Some of these things will still be needed, but strive for being Efficient and be mindful about the need to keep our velocity high.

For example, if you feel confident that what you're shipping is something that won't result in usability issues, effectively meets a customer need, and/or won't result in workflow issues, then we should have the confidence to move ahead with it without conducting research.

If you can skip a process to gain velocity, consider doing that. We may not be able to apply this rule for GA features, though. Be mindful about the impact of skipping process on customers when you are working on Beta feature. If you are working on an Experimental feature, skipping some processes to gain velocity is recommended after thoughtful consideration.

If you have questions about whether a process is hindering efficiency, is outdated, doesn't scale well, and can be skipped or automated, talk to your manager. When you skip a process, document this decision somewhere (a comment, an issue) and ping people for whom this information might be useful.

Embrace risk

When you are working on Experimental and Beta features it is okay to take some risks. We have many useful tools and automation available that can allow us to take some additional risks to retain high velocity when delivering Experimental features:

1. You can use Feature Flags.
1. You can implement circuit breakers.
1. Use design patterns that increase code isolation.

The more isolated your feature is from the rest of GitLab, the more risks you can take, because it is less likely that a problem in your Experimental feature will affect Generally Available features, or a whole product, in a negative way.

There is one exception for this rule: availability and security. Your code should be always as secure as possible and should not bring GitLab.com down, regardless of the maturity model.

Balance maintenance

Balance the need to refactor code and deliver new features. Use our Efficiency principle as a compass.

Be proactive

If there is a process that has a negative effect on our velocity, be proactive about it: speak up and collaborate with your manager on how can we improve it.

We should always be able to explain why a process exists, and if we're unable to do it, maybe it is "a process for process sake", maybe it is a process of low value, or it does not fit our current scale or our strategic goals anymore. If that is the case perhaps the process could be eliminated, fixed, or automated.

Predictability

Prioritizing technical decisions

Please see the Product Management section that governs how they prioritize work, and also should guide our technical decision making.

Despite the high priority of velocity to our project and our company, there is one set of things we must prioritize over it: GitLab availability & security. Neither we, nor our customers, can run an Enterprise-grade service if we are willing to risk users' productivity and data.

Our hundreds of Engineers collectively make thousands of independent decisions each day that can impact GitLab.com and our users and customers there. They all need to keep availability and security in mind as we endeavor to be the most productive engineering organization in the world. We can only move as fast as GitLab.com is available and secured. Availability of self-managed GitLab instances is also extremely important to our success, and this needs to happen in partnership with our customers' admins (whereas we are the admins for GitLab.com).

For security, we prioritize it more highly by having strict SLAs around priorities labels with security issues. This shows a security first mindset as these issues take precedence in a given timeframe.

Leverage Established Processes

We develop and document processes that are designed to help us in achieving better Results.

For example, the following processes exist to reduce the risk and improve code quality:

- Feature Flags Usage
- Code Review Guidelines

We find opportunities for improvement through analyzing metrics to identify trends, hosting retrospectives (e.g. Group Retrospectives, Iteration Retrospectives), performing Root Cause Analyses, and receiving feedback from team members. Team members are encouraged to identify opportunities to improve our processes and propose solutions, examples of this could be an MR or an issue describing these opportunities.

Everyone can contribute by proposing new processes and improving upon existing processes.

Quality

It is important to remember that quality is everyone's responsibility. Everything you merge to master should be production ready. Familiarize yourself with the definition of done.

Release when it's ready

Our releases page describes our two main release channels:

1. Self-managed users use a monthly self-managed release.
1. GitLab.com uses auto-deploy releases.

As the first of these is a monthly release, it's tempting to try to rush to get something in to a monthly self-managed release. However, this is an anti-pattern. Most issues don't have strict

due dates. Those that do are exceptions, and should be treated as such.

Due date pressure logically leads to a few outcomes:

1. People are at increased risk of burnout.
1. We may compromise on our definition of done.
1. We cut scope.
1. We miss the due date.

Only the last two outcomes are acceptable as a general rule. Missing a 'due date' in the form of an assigned milestone is often OK as we put velocity above predictability, and missing the monthly self-managed release does not prevent code from reaching GitLab.com.

For these reasons, and others, we intentionally do not define a specific date for code to be merged in order to reach a self-managed monthly release. The earlier it is merged, the better. This also means that:

1. We don't want merge request authors to work extra hours or otherwise rush to meet a due date.
1. We don't want reviewers and maintainers to be put under pressure to do anything other than meet the regular SLOs.

If it is essential that a merge request make it in a particular release, this must be communicated well in advance to the engineer and any reviewers, to ensure they're able to make that commitment. If a severe bug needs to be fixed with short notice, it is better to revert the change that introduced it than to rush, or even to delay the release until the fix is ready.

In general, there is no need to change any behavior close to the self-managed release.

Dogfooding

We dogfood everything. Based on our product principles, it is the Engineering division's responsibility to dogfood features or do the required discovery work to provide feedback to Product. It is Product's responsibility to prioritize improvements or rebuild functionality in GitLab.

Dogfooding Antipatterns

An easy antipattern to fall into is to resolve your problem outside of what the product offers. Dogfooding is not:

1. Building a bot outside of GitLab.
1. Writing scripts that leverage the GitLab API (if the functionality is on our roadmap and could be shipped within the GitLab Project).
1. Using a component of GitLab that is not part of our components or managed apps.
1. Using templates or repos that are not part of the default UI (having to type or copy-paste to add them).
1. Not deprecating the non-GitLab product or part as you introduce the functionality in GitLab, we need to eat the dogfood, not just serve it.

1. Using it for something that isn't crucial or essential.
1. Delaying until every single feature you use now is in the GitLab product.
1. Not starting it because using a non-GitLab and GitLab product at the same time introduces overhead.
1. Trying to move the most complex part first, instead please iterate and move the simplest first.
1. Building it outside of GitLab because "It is just a prototype", "We're not sure it will work", or "We're not sure it is better than the existing functionality"

Dogfooding Process

Follow the dogfooding process described in the Product Handbook when considering building a tool outside of GitLab.

Code Quality and Standards

We need to maintain code quality and standards. It's very important that you are familiar with the Development Guides in general, and the ones that relates to your group in particular:

- UX Guides
- Backend Guides
- Frontend Guides
- Database Guides

Please remember that the only way to make code flexible is to make it as simple as possible:

```
<blockquote class="twitter-tweet" data-lang="en"><p lang="en" dir="ltr">A lot of programmers make the mistake of thinking the way you make code flexible is by predicting as many future uses as possible, but this paradoxically leads to less flexible code.<br>The only way to achieve flexibility is to make things as simple and easy to change as you can.</p>&mdash; Nearby Cats (@BaseCase) <a href="https://twitter.com/BaseCase/status/1085686616499183616?refsrc=twsrc%5Etfw">January 16, 2019</a></blockquote><script async src="https://platform.twitter.com/widgets.js" charset="utf-8"></script>
```

UI consistency and maintainability across Rails and VueJS Frontend

Part of our engineering culture is to keep shipping so users and customers see significant new value added to GitLab.com or their self-managed instance. To support rapid development, we choose pragmatically the right technology. As each view is unique, we should equally respect our HAML and Vue codebase and make an educated choice per view as to which framework will enable the most consistency and maintainability.

It's important to keep in mind that when building complex applications, there are many factors to consider such as the fully planned feature to avoid situations where we build an MVC in HAML only to later need to re-write it in Vue due to growing complexity.

To promote visual consistency and an accessible UI, we should always aim to use simple and reusable UI components provided by the GitLab UI component library both in Vue and HAML views. We implement GitLab UI components based on our Pajamas design system and currently these are

mostly in Vue, however, we provide adapters that allow us to use a few simple components in HAML as well.

If a GitLab UI component is not available on HAML due to its intrinsic complexity, this is a sign that you should implement your feature using Vue instead.

When to use VueJS

- You are looking to use an already established component from the (gitlab-ui) library.
- You are building a complex feature that will require multiple complex components which are connected to provide interactivity in a single page application.
- You find yourself enhancing a HAML view's behavior with vanilla JavaScript / jQuery (reactive or asynchronous behavior).
- You require medium or larger client-side interactivity.
- You want to build a realtime application.

When to use Rails

- You require a simple, static view.
- You do not need to use multiple components or state management.
- You do not need to use any of the complex components from the (gitlab-ui) library (this does not extend to simple components which can be used in HAML such as labels).

Complex Components

A complex component denotes a type of component that cannot be used easily in our HAML files. This might be due to in-built state management, CSS or dynamic behaviour that rapidly becomes a maintainability burden inside HAML. An example of such a component would be our Table component.

- backend
- frontend
- use GraphQL everywhere
- the importance of velocity
- engineering proposed initiative

SECTION: engineering/development/required-approvals.md

Overview

There are specific scenarios we are identifying that will require additional approval before moving forward. At GitLab we value freedom and responsibility over rigidity, however in the examples requiring approval section below we outline which decisions will need to go through the approval process before proceeding.

Approval Process

Each section requiring approvals will have a considerations section. If you answered yes to all

of the questions in the considerations section, then you will need to get approval for your proposal before proceeding with implementation. Steps for the approval process:

Submit an issue with your design proposals

Make the issue visible to the CEO and all Fellow Engineers.

Label with ~"CEO Interest"

You will need to gain approval from the CEO and one Fellow Engineer

Use the issue created in the first step to coordinate how to gain approval

Examples Requiring Approval

Proposing a separate database

As we've learned from our discussions around defining the container registry database schema there are times when it makes sense to separate the data out into its own database. The following sections will describe what data points you should consider and how you would move forward in gaining approval to create a separate database.

Considerations

Was the feature written in a completely separate code base? Again, using the Container Registry example, this was written in Go and is separate from the main GitLab Rails application.

Only do a separate database when it has a completely separate codebase, and that is the only codebase querying the data. For example, Analytics Instrumentation isn't fit for it since the Rails codebase touches it.

Will the feature incur a heavy write load? Writes are harder to scale than reads who can be scaled with replicas.

Will the feature not need any access to the main Rails database? Example, the data is completely isolated from the main Rails database and will not need to share data across databases.

Are there existing APIs that we need to support? For example, the container registry has a full set of APIs (<https://docs.docker.com/registry/>) that we already use in the existing GitLab Rails application to retrieve tags, etc.

Proposing the creation of microservices

The trouble with microservices is not a new topic to GitLab. With our company strategy of a Flywheel with two turbos we default to delivering GitLab as a single application. However, we are not opposed to adopting the right architecture to solve our problems at scale, in this case microservices. When submitting your proposal for the creation of microservices for approval please be sure to include the following considerations.

Considerations

Given that our direction indicates we are SaaS First how will this change improve our SaaS first offering?

How can these improvements be measured going forward?

How will this impact our self-managed customers?

Can this problem be solved in our current monolithic environment?

Are the ongoing deployment considerations listed in your proposal?

SECTION: engineering/development/secure-coding.md

Secure Coding Training Guidelines

It is essential that all developers are aware of secure coding best practices and refresh this knowledge periodically.

Why

It reduces the chances of security issues being introduced into GitLab.

Some developers want to learn secure coding as a part of their ongoing learning.

It may help to make customers and auditors more comfortable with the security of our product.

Background

Secure coding training was done in late July of 2019.

Secure coding is an onboarding task for new developers.

What challenges does this address?

We do not track who has taken the training to gauge our security risk.

We do not encourage developers to do the training if they have not previously taken the course.

We do not have a plan to motivate GitLab Team Members to refresh their knowledge of secure coding periodically.

Considerations

We want to encourage GitLab Team Members to take this training and periodically refresh their knowledge of it because they believe it is relevant to them and the company. We do not want to force it on them.

We want to track how many GitLab Team Members have taken the training / refreshed their knowledge on it to encourage full participation; however, we do not want to make it feel punitive to them if they have not yet done so.

We want to avoid burdening engineers with the tracking of these activities.

The Plan

February 2020

Create an issue and list each manager and the number of direct reports that are engineers in a table: 6406

Ask managers to update the issue with how many engineers on each team have indicated that have taken the training.

Ask managers to encourage those who have not taken the training to do so.

We chose to have engineering managers track this for their teams vs. using the acknowledgment process as it is believed it will cause it to be better received.

April 2020

Create a new issue and list each manager and the number of direct reports that are engineers in a table.

Create an OKR for Q2 to have $\geq 90\%$ of engineers acknowledge that they have taken the secure training class by the end of Q2 (end of July).

July 2020

Ask managers to update the table with the number of direct reports that have taken the training.

Update OKR based on results.

2021

Repeat the OKR / tracking process to take into account new engineers and those who did not take the training in 2020.

Create a short refresher course on secure coding for those who took the full course previously.

SECTION: engineering/development/fulfillment/_index.md

- Direction
- Groups
- Team

Vision

A high performing team who provides our customers with a world-class buyer experience through the products we build. Our team strives to build an experience that is delightful, performant, trustworthy, and reliable.

Mission

Fulfillment focuses on improving our capabilities and metrics in the following areas:

- Platform: Team
- Provision: Team
- Subscription Management: Features
- Utilization: Team

Direction

In addition to the Fulfillment Product Direction, the Fulfillment Development Sub-department strives to:

- Increase the reliability and availability of our Fulfillment infrastructure
- Make foundational technical improvements to the architecture and data models of the Fulfillment systems
- Improve the developer experience of Fulfillment engineers through better tooling and documentation

Team members

See <https://handbook.gitlab.com/handbook/product/categories/fulfillment-section> for a list of team members for the Fulfillment section and sub-groups.

Stable counterparts

Sales & Go-To-Market

```
{{< member-and-role-by-gitlab "cnodari" "jrabbits" "Gsodhi" >}}
```

Finance & IT

```
{{< member-and-role-by-gitlab "smccauley" "annapiaseczna" "andrewmurray" "smundy" "lmendonca2" >}}
```

Support Engineering

```
{{< member-and-role-by-gitlab "jlyttle" "mdunninger" "kslaats" >}}
```

Office of the CEO

```
{{< member-and-role-by-gitlab "ipedowitz" >}}
```

Product Technical Program Management

```
{{< member-and-role-by-gitlab "cersoz" >}}
```

Project management process

- In accordance with our GitLab values
- Transparently: nearly everything is public, we record/livestream meetings whenever possible
- We get a chance to work on the things we want to work on
- Everyone can contribute; no silos

SAFE

Working in SAFE manner at GitLab is everyone's responsibility. We along with our stable counterparts in Sales and Billing contribute to an area of the product that potentially could encounter sensitive or financial information, and could have an effect on the business as a whole. Therefore, it's important for Fulfillment team members to ensure that the SAFE epics, issues, videos, MRs, and other artifacts we produce are kept confidential.

On occasion, it may be prudent to include language like the following to the description of

public issues where potentially SAFE discussions are happening.

- > This page may contain information related to upcoming products, features and functionality.
- > It is important to note that the information presented is for informational purposes only,
- > so please do not rely on the information for purchasing or planning purposes. Just like with
- > all projects, the items mentioned on the page are subject to change or delay, and the development,
- > release, and timing of any products, features, or functionality remain at the sole discretion of
- > GitLab Inc.

Similarly, not all information should be included in the public handbook. Instead, use the private internal handbook for this SAFE information.

Please reference this documentation about promising features in future versions for more information.

Planning

We plan in monthly cycles in accordance with our Product Development Timeline. Release scope for an upcoming release should be finalized by the 1st.

On or around the 26th: Product meets with Engineering Managers for a preliminary issue review. Issues are tagged with a milestone and are estimated initially.

Planning Issues

- Subscription Management
- Fulfillment Platform
- Provision
- Utilization

Intake Request

To request work to be added to the Fulfillment Roadmap, please open an issue by following this link and tag one of the Fulfillment Product Managers.

A Fulfillment Product Manager will be assigned to review the request and start evaluating the scope and impact. This may take some time depending on team capacity. Once evaluated, the PM will:

1. Create a new Epic, capturing relevant details from the intake issue.
1. Add this new Epic to the Fulfillment roadmap.
1. Update the intake issue description with a link to the new epic.
1. Close the intake issue.

We strictly adhere to this Intake Request process to ensure we capture the full details of a request before actioning on it. Once our Product Managers receive this request, we'll work with the requestor to ensure we have a full understanding of the problem and requirements. After this, we will prioritize your request.

Prioritization

We follow our prioritization framework including cross-functional prioritization guidelines to prioritize our backlog on a monthly basis. R&D teams use a variety of inputs and sensing mechanisms, including SLAs, OKRs, L&R Support Priority Issues list for Fulfillment, technical debt, and more to set priorities.

We aim to maximize business value by balancing short-term, mid-term, and long-term investments across new initiatives and technical debt. At GitLab, we target an overall company allocation of 60% for new product development and 40% for technical debt and maintenance. However, this 60/40 split is flexible and may vary across different groups and sections over time.

In the Fulfillment section, we adopt the 60/40 ratio as a guideline to ensure a well-balanced approach to our OKR-focused work as well as making sure we prioritize enough Engineering initiatives to improve our reliability and efficiency. We remain adaptable in our planning and execution, avoiding a rigid adherence to these percentages. Instead, we prioritize work based on a comprehensive assessment of company needs, which is reflected in our OKRs. We leave it to each group to find the right balance in their OKRs. If an agreement can't be reached within the group, it should be escalated to the section-level to be decided.

For our monthly milestone planning at the group level, we optimize activities to align with our OKRs, remaining flexible and allowing deviations from the 60/40 split. We also make sure to complete within SLA any forced prioritization, as necessary, per our prioritization framework.

Every team uses the monthly prioritization template for cross-functional dashboard reviews every month.

Fulfillment stage dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "SECTIONLABEL"="fulfillment" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "SECTIONLABEL"="fulfillment" >}}
{{< /tableau >}}
```

Subscription Management team dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="subscription management" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
```

```
{{< tableau/filters "GROUPLABEL"="subscription management" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}  
  {{< tableau/filters "GROUPNAME"="subscription management" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="subscription management" >}}  
{{< /tableau >}}
```

Provision team dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="provision" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}  
  {{< tableau/filters "GROUPLABEL"="provision" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}  
  {{< tableau/filters "GROUPNAME"="provision" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="provision" >}}  
{{< /tableau >}}
```

Utilization team dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="utilization" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}  
  {{< tableau/filters "GROUPLABEL"="utilization" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="utilization" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="utilization" >}}
{{< /tableau >}}
```

Fulfillment Platform team dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="fulfillment platform" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="fulfillment platform" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="fulfillment platform" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="fulfillment platform" >}}
{{< /tableau >}}
```

Estimation

Before work can begin on an issue, we should estimate it first after a preliminary investigation. This is normally done in the monthly planning meeting.

| Weight | Description (Engineering) |

| ----- | ----- |

| 1 | The simplest possible change. We are confident there will be no side effects. |

| 2 | A simple change (minimal code changes), where we understand all of the requirements. |

| 3 | A simple change, but the code footprint is bigger (e.g. lots of different files, or tests effected). The requirements are clear. |

| 5 | A more complex change that will impact multiple areas of the codebase, there may also be some refactoring involved. Requirements are understood but you feel there are likely to be some gaps along the way. |

| 8 | A complex change, that will involve much of the codebase or will require lots of input

from others to determine the requirements. |

| 13| A significant change that may have dependencies (other teams or third-parties) and we likely still don't understand all of the requirements. It's unlikely we would commit to this in a milestone, and the preference would be to further clarify requirements and/or break in to smaller Issues. |

In planning and estimation, we value velocity over predictability. The main goal of our planning and estimation is to focus on the MVC, uncover blind spots, and help us achieve a baseline level of predictability without over optimizing. In general departments at GitLab aim for [70% predictability] but in the Fulfillment sub-department we aim for 80% predictability since our work is typically cross-functional and we need to be in lockstep with other departments.

- If an issue has many unknowns where it's unclear if it's a 1 or a 5, we will be cautious and estimate high (5).
- If an issue has many unknowns, we can break it into two issues. The first issue is for research, also referred to as a Spike), where we de-risk the unknowns and explore potential solutions. The second issue is for the implementation.
- If an initial estimate is incorrect and needs to be adjusted, we revise the estimate immediately and inform the Product Manager. The Product Manager and team will decide if a milestone commitment needs to be adjusted.

Estimation Template

The following is a guiding mental framework for engineers to consider when contributing to estimates on issues.

markdown

Refinement / Weighting

<!--

Ready for development means replying yes to the following questions:

- Is this issue sufficiently small enough? If not, break it into smaller issues
- Is it assigned to the correct domain (e.g. frontend, backend)? If not, break it into two issues for the respective domains
- Is the issue clear and easy to understand? If not, try asking further clarification questions and update the description once they are received

If more than 2 MRs are needed, consider adding a table like the following to the description (e.g. under Implementation plan).

| Description | MR |

| - | - |

|||

It will help track the status.

-->

- [] Ready for development

- [] Weight is assigned
- [] Number of MRs listed
- [] Needs testing considerations
- [] Needs documentation updates

Reasoning:

<!--

Add some initial thoughts on how you might break down this issue. A bulleted list is fine.

This will likely require the code changes similar to the following:

- replace the hex driver with a sonic screwdriver
- rewrite backups to magnetic tape
- send up semaphore flags to warn others

Links to previous examples. Discussions on prior art. Notice examples of the simplicity/complexity in the proposed designs.

-->

Picking something to work on

Engineers can find and open their team's planning issue to check their milestone board and begin working first on those with the deliverable label.

It's possible for engineers to pick any of the remaining issues for the milestone once the deliverables are done. If the engineer has no preference, they can choose the next available issue from the top.

Workflow

We generally follow the Product Development Flow and use the workflow labels as defined there.

Generally speaking, issues are in one of two states:

- Discovery/refinement: we're still answering questions that prevent us from starting development,
- Implementation: an issue is waiting for an engineer to work on it, or is actively being built.

Basecamp thinks about these stages in relation to the climb and descent of a hill.

While individual groups are free to use as many stages in the Product Development Flow workflow as they find useful, we should be somewhat prescriptive on how issues transition from discovery/refinement to implementation.

User Experience

We strive to provide excellent usability in all of our workflows, creating a balance between

user and business needs. Product Designers work closely with Product Managers and Engineers.

How we work

- We follow the Product Designer workflows and UX Researcher workflows described in the Product Design section of the handbook.
- We measure our progress using UX Scorecards and Category Maturity Scorecards.
- We prioritize the most important projects every quarter, and Fulfillment product designers support projects instead of Groups.
- We use [UX] issues as the SSOT for designs. Implementation issues should link to the [UX] issue for the design details to maintain the SSOT.
- We use labels to track our issues:
 - UX, devops::fulfillment, section::fulfillment and group::.
 - workflow:: labels to indicate where the issue is in the product development flow
 - UX Problem Validation and UX Solution Validation for research efforts
 - design weight:: for UX issue weights

Product Designer focus areas

The Fulfillment team is Project focused, and many projects span Groups, leading to gaps in the user experience or frequent designer borrow requests. In order to avoid this, we will assign designers to project areas, and revisit quarterly during OKR planning.

- Priorities for the Fulfillment UX team are documented in a prioritization issue.
 - Priorities should be reviewed by the team quarterly, or when a new priority project is identified.
- Anyone can propose projects identified for design focus in the issue. Product Managers should work with Product Designers and the Product Design Manager to rank the priorities. If multiple PMs are contributing to a project, one should be designated for UX planning.
 - If we had to make a priority call to have projects that will go forward without design support, list those decisions in the issue.
- Designers can still pick up issues outside their assigned projects. These should be crucial UX issues such as SUS Impacting issues or bugs. Issue weights can be used to discuss trade-offs when needed.
- Anyone can use the sfulfillmentux Slack channel to ask for assistance.

Best practices

- To manage workload, designers should generally be assigned to no more than one large and 1 small/medium project at a time, or 3-4 small/medium projects (or the equivalent in issue weights).
- Designers should use their best judgement and collaborate with their teams to decide which meetings to attend. Designers aren't expected to attend team sync meetings for multiple teams at the same time.
- Product Designers should be assigned UX MR reviews for the projects they're supporting.
 - If UX review is needed on an MR that is part of a project without an assigned designer, post the request in the sfulfillmentux Slack channel. UX MR review requests in the Slack channel will be picked up based on bandwidth.

UX issue management and weights

We use separate [UX] issues for medium or large projects that will take more than one dev issue to implement (e.g., end-to-end flows, complicated logic, or multiple use cases / states that will be broken down by engineering into several implementation issues). UX issues should be prefixed with [UX].

If the work is small enough that implementation can happen in a single issue, then a separate [UX] issue is not needed, and the designer should assign themselves to the issue and use workflow labels to indicate that it's in the design phase.

- [UX] issues are the SSOT for design goals, design drafts, design conversation and critique, and the chosen design direction that will be implemented. Implementation issues should link to the designs in the [UX] issue as the SSOT.
- Product requirement discussions should continue to happen in the main Issue or Epic as much as possible.
- When the Product Designer wants to indicate that the design is ready for ~"workflow::planning breakdown", they should apply this label to their issue, notify the PM and EM, and close the issue.
- Issue weights should follow the UX Department's definitions and are applied using the ~"design weight:" label.

Approving and merging the work

We have approval rules enabled in CustomersDot, so every MR that targets main needs at least one approval (different from the author/committer).

The MRs must meet the following criteria:

- They need to have at least two different reviewers, including one from a maintainer. However, for trivial MRs with no changes in logic one reviewer is sufficient and you can skip the initial review for such changes (for example, minor dependency updates, test fixes, and plain reverts).
- They must receive at least one approval.
- A maintainer's review is required.

In addition to the approval rules, MRs may require additional reviews as suggested by the Danger bot:

1. Modifications to the DB require database reviewer and maintainer approval
1. Security-related issues (such as changes to authentication) require a Security review
1. Changes to the SFDC APIs require review by the Sales team
1. Changes to the Zuora APIs require review by the EntApps team
1. Changes to the user experience require a UX Review.

Weekly async issue updates

Every week, each engineer is expected to provide a quick async issue update by commenting on their assigned issues using the following template:

markdown

<!--

Please be sure to update the workflow labels of your issue to one of the following (that best

describes the status)"

- ~"workflow::In dev"
- ~"workflow::In review"
- ~"workflow::verification"
- ~"workflow::blocked"

-->

Async issue update

1. Please provide a quick summary of the current status (one sentence)

-

1. How confident are you that this will make it to the current milestone?

- ☐ Not confident
- ☐ Slightly confident
- ☐ Very confident

1. Are there any opportunities to further break the issue or merge request into smaller pieces (if applicable)?

- ☐ Yes
- ☐ No

1. Were expectations met from a previous update? If not, please explain why.

- ☐ Yes
- ☐ No,

1. Are the related issues up to date? Please link any missing issues in the epic that could be linked to this issue

- ☐ Yes
- ☐ No

/healthstatus [ontrack, needsattention, atrisk]

We do this to encourage our team to be more async in collaboration and to allow the community and other team members to know the progress of issues that we are actively working on.

Async projects update

This template is to be used for larger project progress status updates to be shared with leadership & cross-functional partners.

Leadership is interested in the following

1. Is the project progressing? --> Look at % completion week to week
2. Are there blockers that need to be cleared? --> Look at Risks & Blockers
3. When can we expect the next event? --> Look at Key dates

This template is a guideline and feel free to alter it to specific project needs. Before deviating from the template keep in mind that leadership looks at status updates across multiple projects. The more conformity there is between projects the easier it is for leadership to accurately understand status.

markdown

Status update as of XXXX-XX-XX

Summary

1. Key Resources

TBD this section is optional

2. % Complete: X%

3. Status: On Track or Behind (this is determined based on your how your % complete is trending to your key dates -- are you far enough along to hit your key dates?)

4. Key Dates:

Design complete - Milestone XX.X

Development complete - Milestone XX.X

Rolled out in production - XXXX-XX-XX

Risks & Blockers

| Risks & Blockers | Mitigation Approach |

|-----|-----|

| New! | |

Results/Challenges/Learnings

List any type of deliverable, e.g. merged MRs, alignment on solution, copy/designs were completed.

1.

FYI TAG FOLKS

Demos

Some work can take a few milestones to complete. Together with regular async issue updates, it is useful to set demos. Advantages are:

- shorten the feedback cycle
- increase visibility of the work
- improve communication of work and progress

Note that providing demos and recordings is already encouraged to ease the review process in MRs.

For this purpose, a YouTube playlist has been created: Fulfillment Demos (some content is internal only). To upload your demos, you can follow the process described here. Once the video is created, link it to the Fulfillment Demos playlist. Consider adding the following info in the description:

- A link to the Epic/Issue
- A link to the MR (if any)
- The name and link to profile of who uploaded it (optional)

Here's an example (internal only)

Quality

GitLab's Quality is everyone's responsibility.

End-to-end tests - how, when and why to write them

End-to-end tests (often referred to as e2e tests) cover full or partial flows that the end-user will go through. Detailed information on testing levels can be found [here](#).

Some examples of these flows:

- Purchasing a SaaS subscription for a namespace on GitLab.com.
- Purchasing a new subscription for a self-managed GitLab instance.

Since these tests are not fast and more prone to being flaky, prioritization of what to cover with end-to-end tests and what to delegate to lower level tests must be considered.

Fulfillment tests reside both in the GitLab Project and the CustomersDot Project, use the following guide to determine which project to use when writing the tests:

- Choose CustomersDot if the test will require opening and performing actions in the CustomersDot portal, otherwise GitLab will be the correct project.
- In the future, it is planned that CustomersDot will become just a backend service and UI end-to-end tests will reside only in GitLab.

GitLab

The GitLab end-to-end testing guide can be found [here](#)

Planned & automated Fulfillment test cases in the GitLab project can be found [here](#).

CustomersDot

The CustomersDot end-to-end beginners guide can be found [here](#).

Planned/automated test cases in the CustomersDot project can be found [here](#).